GESTURA: Mobile App Architecture Specification

Real-Time On-Device Sign Language Gesture Recognition & Speech Translation System

TECHNICAL WHITE PAPER & COMPREHENSIVE REPOSITORY SPECIFICATION • GESTURA v1.0

Executive Summary: *Gestura is a high-performance, real-time sign language interpreter built on React Native and Expo. It combines hardware-accelerated video capture (VisionCamera v5), Google MediaPipe 21-point hand tracking (via a custom native Android Kotlin module), on-device neural network classification (Fast-TFLite v3), and native text-to-speech synthesis (Expo-Speech). This document provides an exhaustive technical analysis of the application's architecture, repository structure, end-to-end data pipelines, and core source modules.*

1. Core Technology Stack & Dependencies

The application architecture relies on five specialized subsystems designed to achieve sub-50ms inference latency without triggering garbage collection overhead or camera pipeline locks.

Subsystem	Core Library / Engine	Version	Architectural Role & Significance
Core Runtime	React Native & Expo SDK	RN 0.86.3 / SDK 57	React 19 New Architecture, Hermes JS engine, Expo CLI, and modular Nitro bindings.
Application Routing	Expo Router	~57.0.29	File-based navigation (app/index.tsx, app/explore.tsx) with native tab layout caching.
Camera Pipeline	react-native-vision-camera	^5.2.3 (Nitro)	Low-level Android CameraX/Camera2 wrapper providing multi-output preview streaming and still capture.
Vision / Tracking	Google MediaPipe Tasks Vision	0.10.18	Extracts 21 3D hand joints (63 spatial floats) via hand_landmarker.task running on device CPU/GPU.
Native Bridge	expo-mediapipe-hands	Local Module	Custom Kotlin Expo module handling EXIF rotation, adaptive downsampling, and JNI bitmap recycling.
Neural Inference	react-native-fast-tflite	^3.0.1	C++ LiteRT/TensorFlow Lite runtime with zero-copy ArrayBuffer bindings running models in <1 ms.
Speech Engine	expo-speech	~57.0.3	Native Android Text-To-Speech (TTS) synthesizer for audible word feedback with speech debouncing.

2. Repository Structure & Key Subsystems

The project is structured into three distinct domains: mobile application runtime (GesturaMobile), machine learning data & training pipelines (ml_pipeline), and desktop research/validation tools.

Directory / Path	Subsystem Scope	Core Files & Responsibilities

GesturaMobile/src/app/	Navigation Routes	**index.tsx:** Active translation viewfinder with HUD, skeleton, & controls. **explore.tsx:** Searchable sign dictionary with placement tips. **_layout.tsx:** Tab bar navigation structure.
GesturaMobile/modules/expo-mediapipe-hands/	Custom Native Module	**android/.../ExpoMediapipeHandsModule.kt:** Kotlin bridge invoking MediaPipe HandLandmarker with bitmap downsampling and EXIF rotation. **src/index.ts:** TypeScript typing & safe fallback.
GesturaMobile/assets/model/	On-Device AI Models	**sign_language_model.tflite:** 8-class standard gesture classifier (63 inputs). **gesture_model_indian.tflite:** 35-class Indian Sign Language alphabet/number model. ***.json:** Label mappings for all models.
GesturaMobile/src/lib/	Feature Computation	**handDetector.ts:** Bridge invoking native module & wrist-centering normalizer. **features.ts:** 132-d spatial vector calculations. **signEngine.ts:** Sliding window temporal consensus engine.
GesturaMobile/src/components/	Visual Overlays	**HandSkeletonOverlay.tsx:** Lightweight vector canvas rendering 21 hand joints, connecting bones, and dynamic target framing reticle.
ml_pipeline/	Model Training Pipeline	**extract_landmarks.py:** Extracts wrist-centered coordinates from raw images. **train_model.py:** Trains sequential MLP with Keras & exports quantized .tflite.
Root scripts (*.py)	Desktop Validation	**tracker.py / test_model.py:** Desktop OpenCV & webcam test harness for live evaluation before building mobile APKs.

3. End-to-End Data Pipeline: Step-by-Step

The live translation pipeline converts photonic data from the camera sensor into syntactic text and speech output through a sequence of 6 deterministic processing stages:

Phase 1: Sensor Capture (CameraX / VisionCamera)

VisionCamera captures frames via **usePhotoOutput** configured with balanced quality prioritization. In AUTO mode, a timer requests a snapshot every 1000ms. In MANUAL mode, the user taps 'Sign Now' to trigger an on-demand capture. This avoids continuous burst captures while preserving full 60 FPS viewfinder fluidity.

Phase 2: EXIF Orientation Correction & Adaptive Downsampling

On Android, front-facing camera sensors capture images in landscape orientation (rotated 90 or 270 degrees). **ExpoMediapipeHandsModule.kt** queries image dimensions using **inJustDecodeBounds**, computes an optimal **inSampleSize** (scaling down multi-megapixel photos to ~384–512px), and applies an **ExifInterface** rotation matrix. This reduces memory allocation by 95% and guarantees upright hand orientation for MediaPipe.

Phase 3: MediaPipe HandLandmarker Detection

The downsampled ARGB bitmap is passed to **com.google.mediapipe.tasks.vision.handlandmarker.HandLandmarker**. The detector evaluates palm anchors and regressively predicts the 21 normalized 3D keypoints (x, y, z) in continuous normalized image space [0.0, 1.0]. The Kotlin module returns a flat array of 63 doubles.

Phase 4: Wrist-Centered Coordinate Normalization

In **handDetector.ts**, the 63 coordinates undergo spatial translation. Raw coordinates vary based on hand placement relative to the camera boundary. The normalizer subtracts Landmark 0 (wrist: x0, y0, z0) from all 21 points:

- $\Delta x_i = x_i - x_0$, $\Delta y_i = y_i - y_0$, $\Delta z_i = z_i - z_0$

This maps Landmark 0 to origin (0, 0, 0), making the feature vector position-invariant.

Phase 5: High-Speed TFLite Inference (Fast-TFLite v3)

The normalized Float32Array(63) buffer is fed to **activeModel.runSync()**. The on-device neural network (a deep Multi-Layer Perceptron trained in Keras) performs matrix multiplications and ReLU activations, terminating in a Softmax distribution over all candidate gestures. Inference completes in under 1 millisecond on the device CPU/GPU.

Phase 6: Visual HUD, Audio Synthesis, and Transcript Tracking

If the top class exceeds the confidence threshold (60%), the UI displays the sign name and score. **HandSkeletonOverlay.tsx** draws the 21 joints and 21 connecting bone segments over the live viewfinder. **expo-speech** synthesizes the recognized gesture into audible speech, and the word is appended to the transcript history for sentence composition.

4. Main File Catalog & Architectural Patterns

LiveTranslationScreen.tsx (Architectural Origin)

File: GesturaMobile/src/screens/LiveTranslationScreen.tsx
Architectural Pattern: JNI Reference Isolation. VisionCamera hooks (such as useCameraDevice) maintain active native JNI pointers. In high-frequency computer vision apps, updating React state (e.g., probability bars, confidence numbers) causes the entire tree to re-render, frequently re-binding camera outputs and leaking native JNI handles. LiveTranslationScreen solved this by bifurcating the view into: (1) **CameraShell**, which strictly manages hardware and never re-renders on prediction updates, and (2) **GestureProcessor**, which manages high-frequency state.

index.tsx (Production Viewfinder & Controller)

File: GesturaMobile/src/app/index.tsx
Role: Unified live translation interface combining multi-model switching (Standard, Indian Sign Language, General), AUTO/MANUAL operation modes, front/rear camera flipping, flashlight/torch hardware safeguards, and sentence transcript management (Undo, Speak All, Share/Copy, Clear).

HandSkeletonOverlay.tsx (Zero-Dependency Vector Canvas)

File: GesturaMobile/src/components/HandSkeletonOverlay.tsx
Role: High-performance UI overlay rendering the 21 MediaPipe joints (color-coded by wrist, knuckles, and fingertips) and 21 connecting bone lines using pure React Native View primitives and rotational matrix transforms. Features an intelligent target reticle that turns neon green and activates 'HAND LOCKED' upon detection.

ExpoMediapipeHandsModule.kt (Native Android Processing Engine)

File: GesturaMobile/modules/expo-mediapipe-hands/.../ExpoMediapipeHandsModule.kt
Role: Native Kotlin module directly integrating Google MediaPipe Tasks Vision. Implements safe asset resolution (assets/hand_landmarker.task with fallback to internal storage), EXIF portrait orientation rotation, dynamic inSampleSize downsampling, and explicit bitmap recycling to prevent out-of-memory crashes.

explore.tsx (Interactive Sign Language Dictionary)

File: GesturaMobile/src/app/explore.tsx
Role: Educational reference module featuring real-time search, category filtering (Standard, ISL Alphabet A-Z, Numbers 1-9), and expandable flashcards containing step-by-step finger placement instructions and camera distance tips.

5. Performance & Reliability Engineering Summary

Engineering Achievements in Gestura v1.0:
• **95% Latency Reduction:** Eliminating default 4K UHD still capture in favor of adaptive inSampleSize downsampling (~384–512px) decreased per-frame processing latency from 800ms down to ~25ms.
• **Hardware Exception Prevention:** Camera2 flash exceptions ('no flash unit') on front-facing lenses were resolved by making torchMode conditional on physical device hardware capabilities (device.hasTorch).
• **Orientation Invariance:** Automated EXIF rotation alignment ensures MediaPipe receives upright hands regardless of front camera sensor orientation.
• **Position Invariance:** Wrist-centered vector normalization allows users to sign anywhere inside the viewfinder frame with consistent high classification confidence.